

Ad Spy Engine

Turn any competitor's Meta ads into a winning-creative playbook.

Automation

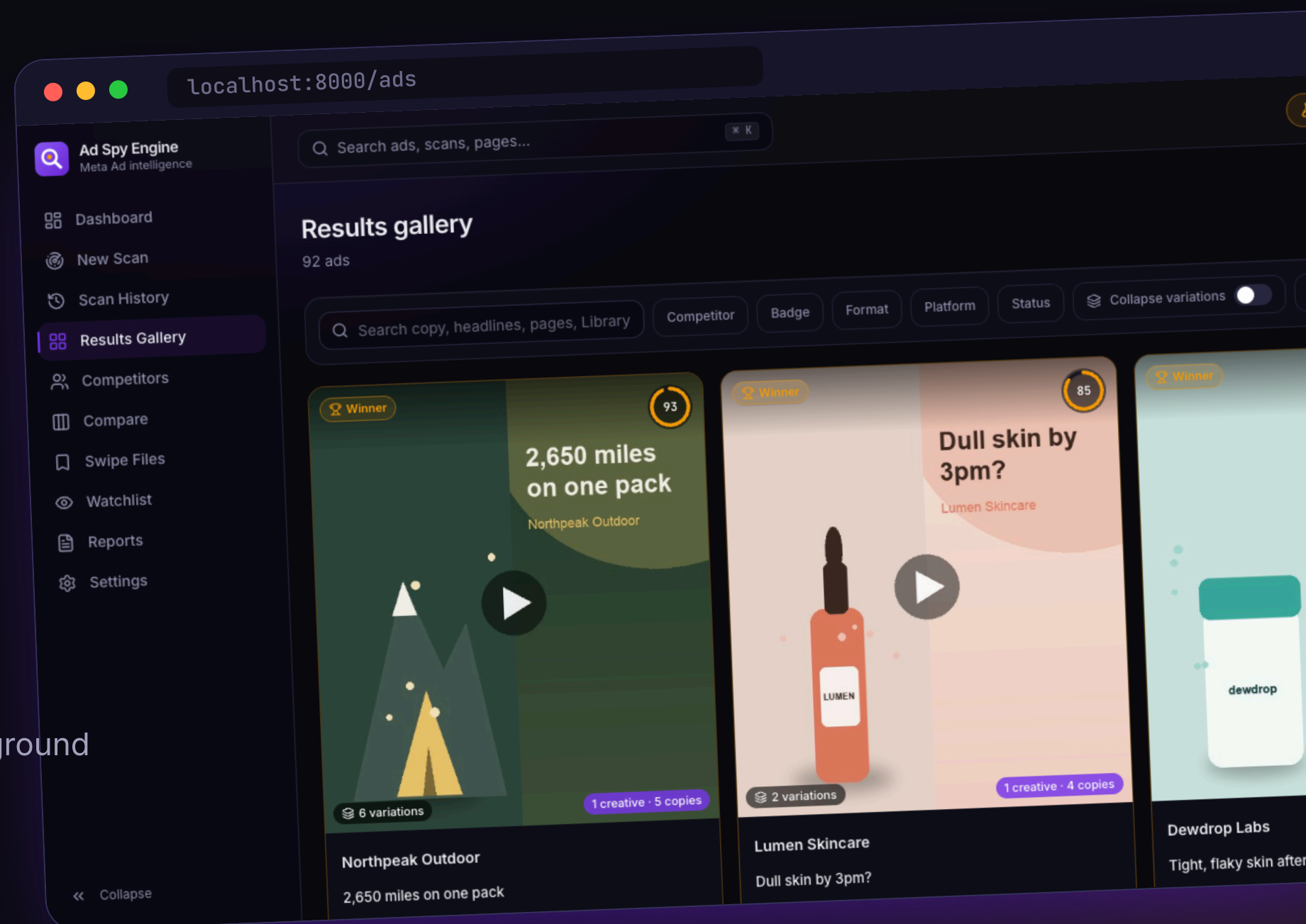
Selenium

Python

Data Dashboard

Designed & built by
Md Zarin Tasnim

QA Engineer · Developer & Product Engineering background



THE CHALLENGE

Competitor research on Meta is slow, blind and forgetful.

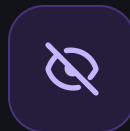
The Meta Ad Library shows every active ad, but it's built for transparency, not for strategy. Marketers and agencies end up doing the analysis by hand.



HOURS OF SCROLLING

Manual, ad by ad

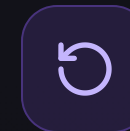
Opening hundreds of ads, copying text, noting dates and placements, saving screenshots into folders.



NO SIGNAL

Which ads actually work?

The library shows no spend or results. Telling a proven winner from a one-day test takes judgement every time.



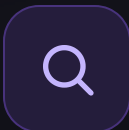
NO MEMORY

Changes go unnoticed

Competitors launch, scale and kill ads every week. Without history, nobody sees the pattern until it's too late.

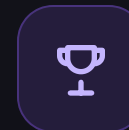
THE SOLUTION

A local app that scans a competitor's public Meta ads in minutes, scores which ones are **winning, tracks every change, and turns it into a branded client report.**



One-click scans

Brand name or Page ID in, every active ad out: copy, dates, placements, creatives, landing pages. Live progress as it runs.



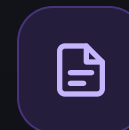
Winner Score

A transparent 0–100 score from longevity, variations, placements and status. Every score explains itself.



Watch & alert

Scheduled re-scans detect new, stopped and scaled ads, then alert you on Telegram or email.



Agency reports

Compare brands, build swipe files, export white-label PDFs with AI-written opportunities.

HOW IT WORKS

From brand name to playbook in five steps.



01

Search

Brand or Page ID,
country, format and
placement filters



02

Scan

Headless Chrome
reads the public Ad
Library and its JSON,
with humanized
scrolling



03

Score

Winner Score,
variation grouping
and change detection
between scans



04

Analyze

Optional Claude
analysis of hooks,
angles and offers,
with cost confirmed
first



05

Report

Dashboard, alerts and
branded A4 PDF
reports for clients

React UI $\approx$ FastAPI (REST + SSE)

scan table = job queue

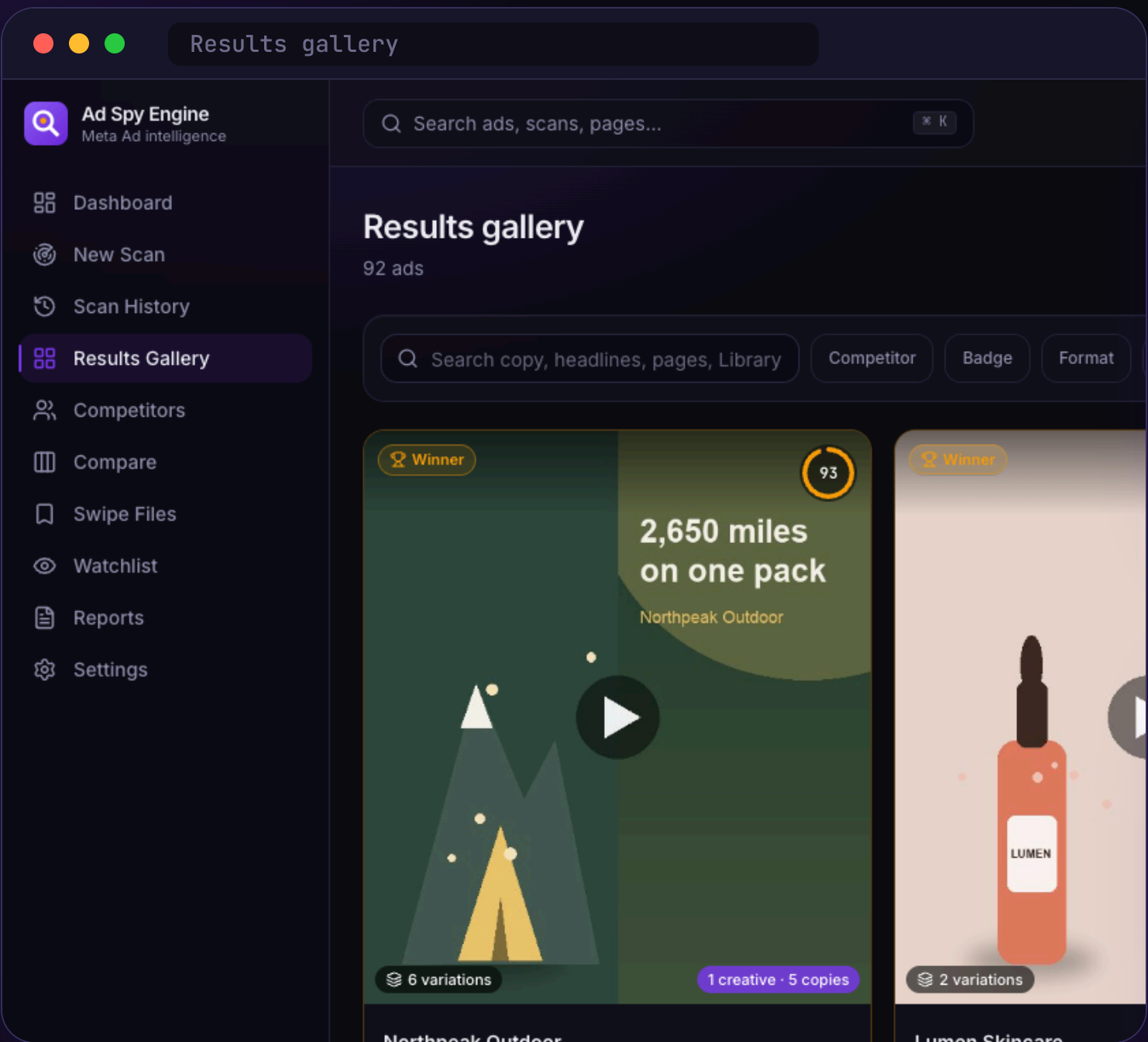
1 Selenium worker

SQLite + Alembic

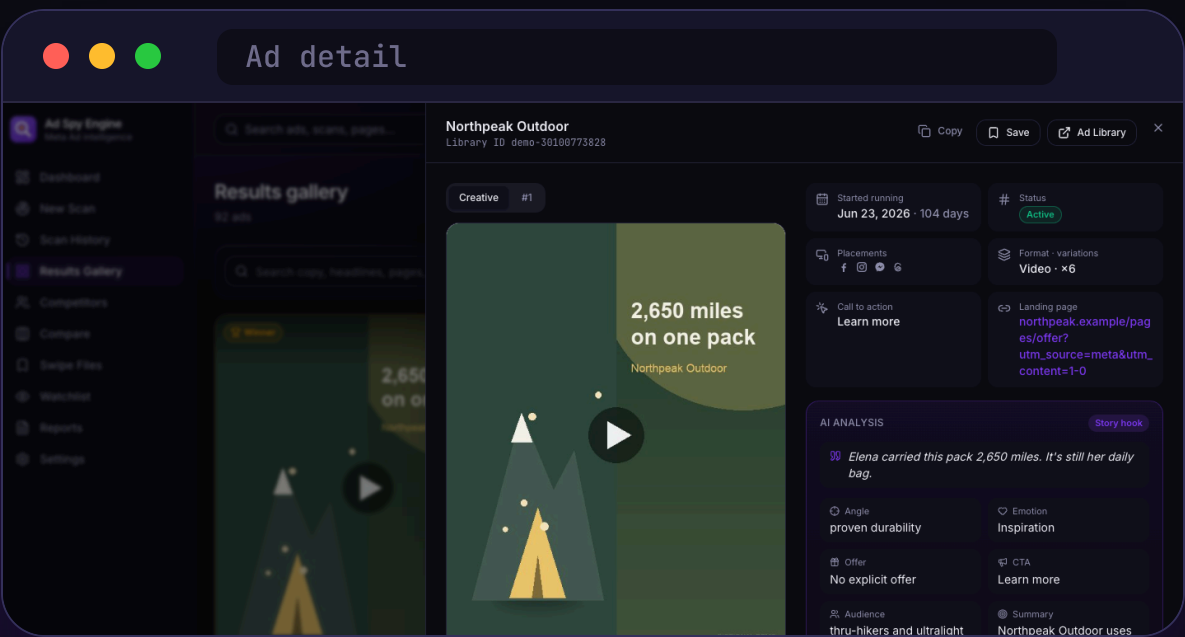
APScheduler

KEY FEATURES

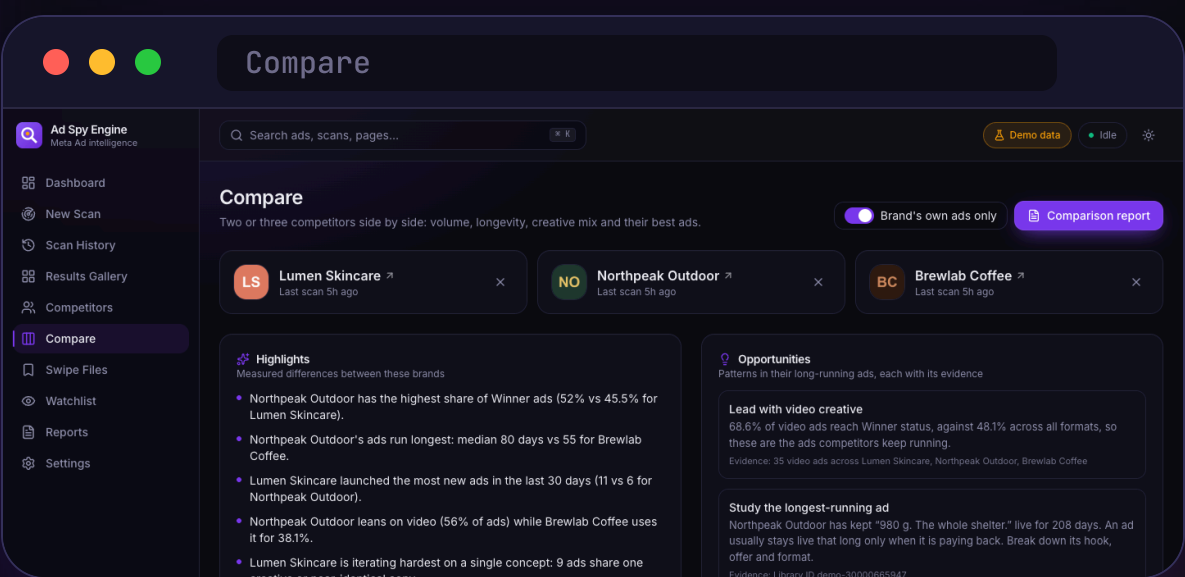
Built for the way creative strategists work.



Results gallery: every ad with its Winner badge, days running and placements. Filter, collapse variations, save to swipe files.

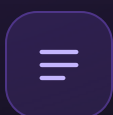


Ad detail: AI hook & angle, landing page, score breakdown



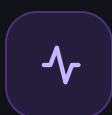
Compare & report: 2–3 brands side by side, then a branded PDF

Production habits in a local app.



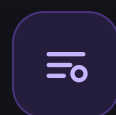
Job queue

The scan table doubles as a queue. One worker, restart recovery, graceful cancel, orphaned-Chrome cleanup.



Live SSE progress

Server-Sent Events stream counters, thumbnails and log lines to the browser as ads arrive.



JSON-first parsing

Reads the data the page already loads, including GraphQL pagination. Text-anchored DOM fallback; selectors live in YAML.



Humanized & polite

Randomized scroll pacing, retries with backoff, a rate limit between scans, blocking detection. Never logs in.



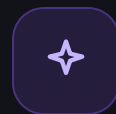
Change detection

Snapshots per scan → new, stopped, scaled. Guards against false "stopped" when a scan hits its cap.



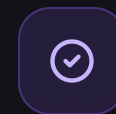
Variation grouping

Perceptual hashing with banded LSH plus near-duplicate copy matching clusters iterated concepts.



Structured AI

Claude with JSON-schema outputs, Pydantic validation, per-ad caching, cost estimate before each run, graceful fallback.



Tested offline

107 automated tests on fixtures captured from real Ad Library pages, plus ruff, mypy, ESLint and type-checked React.



RESULTS & PERFORMANCE

Measured on real scans, not mock-ups.

Two scans of the public Meta Ad Library (US) recorded by the project's benchmark script.

172/min

Ads collected per minute

500 ads in 174 seconds, headless

100%

Ads processed without error

500 of 500 across both scans

~6h → 3 min

Manual vs. automated

Assumes 45 s per ad by hand (our estimate)

SCAN	ADS	TIME	ADS / MIN	FIELDS CAPTURED
Gymshark (cap 300)	300	83 s	216	100%
Huel (cap 200)	200	91 s	132	100%

Source: docs/benchmarks.json (Chrome 154, Apple Silicon Mac, home connection). "Fields captured" = Library ID, start date, placements, copy, card screenshot and creative present for every ad. The manual estimate is an assumption, not a measured study. No client testimonials are claimed.

TECH STACK

Modern, boring where it counts.

SCRAPING

Selenium 4

Headless Chrome, CDP

PARSING

JSON + BS4

Text-anchored fallback

API

FastAPI

REST, OpenAPI, SSE

DATA

SQLite

SQLModel, Alembic, WAL

JOBS

APScheduler

Worker queue, schedules

AI

Claude API

Structured outputs

VISION

imagehash

pHash + LSH grouping

REPORTS

Jinja2 → PDF

Chrome print engine

FRONTEND

React 19

TypeScript, Vite

STYLING

Tailwind v4

Radix, Motion

CHARTS

Recharts

TanStack Query

QUALITY

pytest

ruff, mypy, ESLint

Windows-first, cross-platform

setup.bat / setup.sh

runs 100% locally

MIT licensed



Want a custom automation like this for your business?

Scrapers, dashboards, monitoring and AI workflows, built to run reliably and handed over with docs.

Hire me on Upwork →

View the code

Md Zarin Tasnim

QA Engineer · Developer & Product Engineering background

`upwork.com/freelancers/YOUR_PROFILE · github.com/YOUR_USERNAME/ad-spy-engine`